

FPGA-Based Hardware-in-the-Loop Real-Time Simulation Implementation

Farshideh Kordi*, Christian Barnard*, Paul Fortier*, Amine Miled*

Email : farshideh.kordi.1@ulaval.ca, christian.barnard.1@ulaval.ca,

paul.fortier@gel.ulaval.ca, amine.miled@gel.ulaval.ca

* *Departement of Electrical and Computer Engineering, Université Laval, Québec, QC, Canada.*

Abstract—Hardware-in-the-Loop (HIL) simulators have become an essential tool in the automotive industry as they allow comprehensive testing and validation of electronic control units (ECUs). Although HIL simulations are widely used in the vehicle, aerospace and other industries that rely on electronic control systems, they can also be quite challenging to develop. One of the main challenges of HIL simulators is creating a virtual environment that accurately replicates the behavior of the real-world system. Artificial intelligence (AI) algorithms can be trained on extensive real-world datasets to create more accurate virtual environments of complex systems for HIL simulations. Moreover, latency is a crucial factor in producing a reliable virtual environment for ECU in HIL simulations. FPGA (Field Programmable Gate Array) can help to minimize latency in HIL simulations by providing high-performance computing resources. This paper aims to address these challenges by presenting a machine learning-based HIL simulator design on FPGA. The proposed architecture uses FPGA to accelerate the computations of a temporal convolutional neural network (TCN). The paper also describes how increasing the number of input channels can improve the performance of the TCN model on an FPGA.

Index Terms—Hardware-in-the-Loop (HIL) simulator, electronic control unit (ECU), temporal convolutional networks (TCN), artificial intelligence (AI), anti-lock brake system (ABS)

I. INTRODUCTION

Over the past decade, there has been swift advancement in both the electrification and automation of modern vehicles. An accurate and reliable simulation and testing environment is critical in the development and research of self-driving vehicles. Hardware-in-the-Loop (HIL) simulators can be used to create a virtual environment for testing and validating the ECU control algorithms and system designs before they are implemented into a physical system [1]. Also, a HIL simulation has the ability to prevent significant damage to the physical system, minimize the costs associated with debugging, and ultimately reduce the overall effort required during testing [2]. In the process of HIL testing, the ECU is provided with real-time measurements of observable variables obtained from the simulator, and it subsequently sends control signals to operate the HIL simulation accordingly. HIL testing has the ability to replicate real-world conditions and obstacles encountered by vehicle subsystems, such as braking, suspension, and steering. Field Programmable Gate Arrays (FPGAs) have become increasingly popular in recent years for real-time simulations due to their high processing power and ability to be reconfigured for different applications [3]. Furthermore, the use of FPGAs allows

for parallel computing, enabling real-time simulation models to be computed within the microsecond to nanosecond range. The nanosecond-level update of FPGAs enables high-speed simulations with very low latency, making them a preferred choice for many real-time applications [4]. In [5], [6], FPGAs are used to simulate a variety of systems, including vehicle motion, sensor chips, electric circuits, and high-frequency interfaces. On the other hand, the development of autonomous vehicles demands advanced computing capabilities, cutting-edge algorithms, and deep learning frameworks to enable them to adjust to changing environments [7]. The emergence of artificial intelligence (AI) has led to significant progress in various fields and increased the visibility of these technologies [8]. Neural network (NN) algorithms offer several advantages, including simplifying complex algorithms, reducing computational demands, and improving fault tolerance [9]. Deep neural networks consisting of one-dimensional convolutions and dilation are highly effective for time series and sequence classification and segmentation tasks. These structures are commonly referred to as temporal convolutional networks (TCNs) [10]. Time sequence modeling with artificial neural networks is historically associated with recurrent neural networks (RNNs) [11]. TCNs are a type of neural network which use causal convolutions with dilation to increase the receptive field on the input data exponentially with each layer [12]. Based on causality, the TCN employs a novel causal convolution operation that ensures each output only depends on past and present input values [13]. One of the major advantages of TCNs is their inherent parallelism, which allows for efficient parallel processing of input sequences, leading to faster computation times. TCNs also exhibit greater stability of gradient during training as compared to RNNs [10]. The rest of this paper is organized as follows. Section II gives a brief overview of the HIL simulator design. The hardware implementation of TCN algorithms on FPGAs is described in Section III. Section IV reports results of the FPGA-based model. Conclusions are given in Section V.

II. HIL SIMULATOR DESIGN

The behavior of a truck is emulated with the HIL simulation to be provided as inputs to the ECU. The HIL simulator operates through a loop between the automobile's ECU, which serves as the hardware control system, and a virtual environment. The system's conceptual diagram is illustrated in Fig. 1. In this design the HIL simulator employs a neural network

model to produce more complex signals for the ECU's input. Based on Fig. 1, the technical procedure to implement the proposed architecture is divided into 3 phases:

- 1) The simulation model of the ECU ABS test is developed in the TruckSim simulator and the dataset with ABS and without ABS is extracted from the TruckSim model.
- 2) The TCN Python-based model is used to calculate the weights for the FPGA neural network model. The target files (CSV files) that are generated by the TruckSim simulator are used as datasets for the TCN model in the Python code.
- 3) The TCN FPGA-based model is a prediction module that is used to generate the sensor data for the signal generator module to send to the ECU module. In fact, this module is used to generate signals for the ECU's inputs. The weights which are calculated by the TCN Python-based model are used as the weights model for the neural network model on the FPGA implementation.

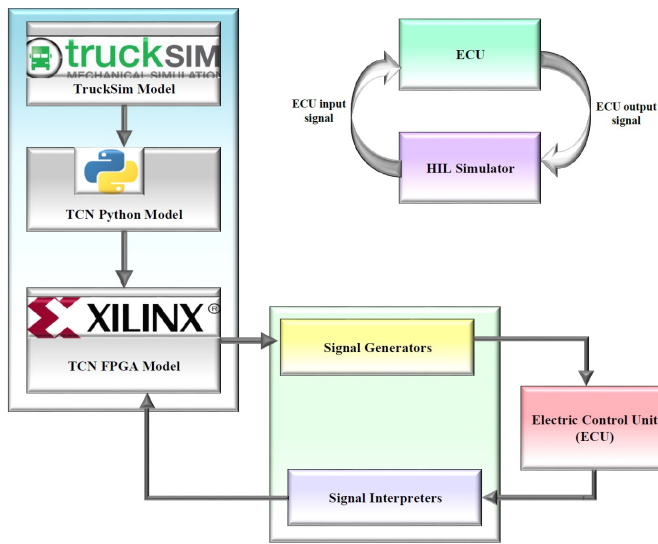


Fig. 1. HIL structure for ECU testing.

III. FPGA IMPLEMENTATION OF TCN REAL-TIME MODEL

The top-level view of the FPGA implementation is depicted in Fig. 2. The weight-access module is used to read the weights model from the TCN Python-model, and the prediction-scheduler module is used for the scheduling prediction scenario in the TCN FPGA-model. Moreover, the outer-communication and Ethernet modules are related to the communication interface of the FPGA to other modules of the HIL simulator. The communication between the predictive module and the system is facilitated through Ethernet, and a custom protocol is used for this purpose. The protocol is designed to work with a central switch, which is also implemented on an FPGA.

There are various types of TCN algorithms. The Encoder-Decoder TCN (ED-TCN) is a type of TCN that uses a

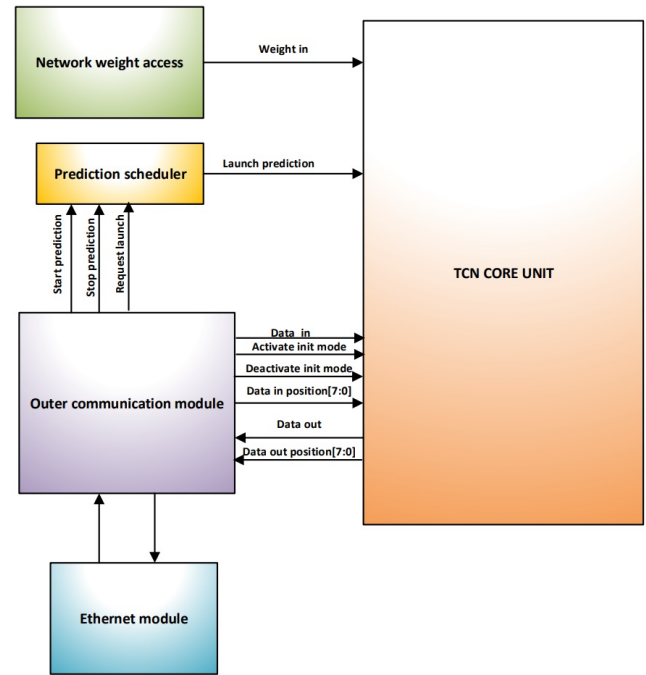


Fig. 2. Architecture of the TCN core implementation on FPGA.

hierarchy of temporal convolutions, pooling, and upsampling in an encoder-decoder architecture to efficiently capture long-range temporal patterns [14]. Dilated TCN is another variant of TCN that uses dilated convolutions instead of pooling and upsampling. Fig. 3 displays the cell structure of the selected architecture, which is based on the ResNet TCN. The ResNet TCN is a type of TCN that consists of a sequence of dilated causal convolutions with residual connections layer [15]. The implemented model uses a single instance of each type of layer, which is shared across the network to reduce the resources needed [16], [17]. For the TCN neural architecture, only three types of layers are required: an addition layer for the residual connections, a ReLu layer, and the fully-connected implementation of the convolutional layers. For normalization, we applied weight normalization to the convolutional filters.

TCN is selected as the predictive model to be implemented. We selected a TCN model containing 64 or 128 input ports, three hidden layers, kernel size of three, a base dilatation factor of three and 17 output ports such as the angular speed of wheels, the lateral forces, the roll angle of the vehicle, etc. The details of the TCN implementation on the FPGA are shown in Fig. 4. The weights model generated by the Python-based TCN model are stored in block-RAM as a model for the FPGA design. The TCN core unit is managed by the inner control module. Each layer in the architecture processes its own sequences and sends its own signals to the TCN module when it has completed the processing and produced its output.

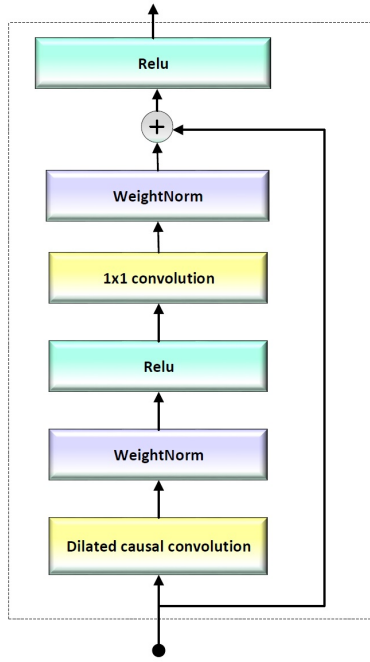


Fig. 3. The structure of the TCN model for each input cell.

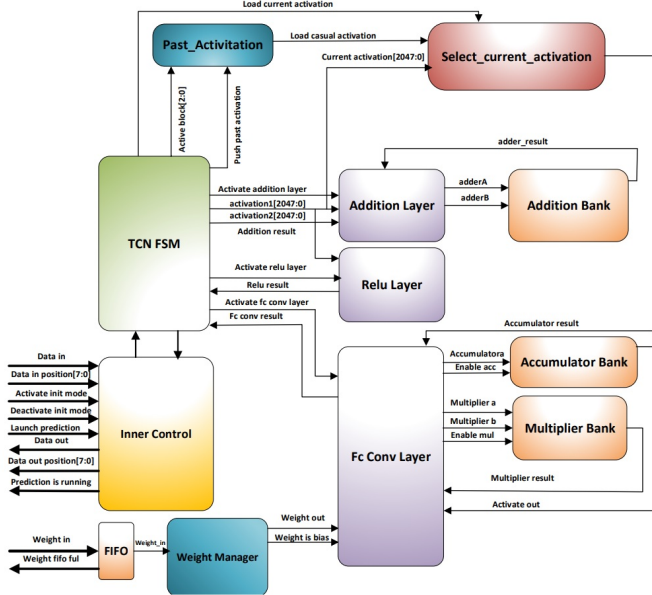


Fig. 4. Block diagram of TCN implementation on FPGA.

IV. RESULTS AND DISCUSSION

The neural network model developed in this paper is trained to predict the ABS sensor parameters required by an ECU to effectively control the braking system of a vehicle. By accurately predicting these parameters, the system can respond quickly to changing conditions, improving the overall safety and performance of the vehicle. The dataset produced by the

TruckSim simulator comprises 2000 truck driving simulations that range from 2 to 10 seconds in duration, totaling about 4 hours of non-consecutive driving. Additionally, the dataset was sampled at a frequency of 60 Hz.

We present some simulation scenarios to compare the result of the TCN Python-based model and the FPGA-based model. First, the FPGA is initialized with defined input values and weights from the TCN Python-based model. The model predicts the output values, there are validation scenarios in which the prediction output is evaluated using a Python-based model and real data from the TruckSim simulator. Second, in the FPGA implementation, a scenario file (CSV file) from the TruckSim dataset is used as a reference file. The FPGA model employs this file along with the weights model to generate the TCN model, and the predictions results are written in a prediction file. The obtained results from the TruckSim simulator, the prediction results from the TCN Python-based model and the TCN FPGA-based model for ABS parameters are presented in Fig. 5. The minimum number of input channels required for the TCN model to generalize its predictions correctly was 64. In the TCN model, each channel refers to the number of feature used in each convolutional layer of the network. Increasing the number of input channels can improve the accuracy and performance of the model, allowing it to capture longer temporal patterns in the input data. In this experiment, based on Table I the number of input channels varied between 64 and 128. Therefore, the sequence input lengths of the two states are changed between 1024 and 2048. The results in Fig. 5 shows that increasing the sequence length performs better on the TCN Python-based and the FPGA-based models. The mean square error loss (MSE loss) is a commonly used loss function in regression problems, which calculates the average of the squared differences between the predicted values and the real values. It is used to evaluate the efficiency of both models in Table I. When using 128 input channels, the MSE loss value decreases compared to when using 64 input channels.

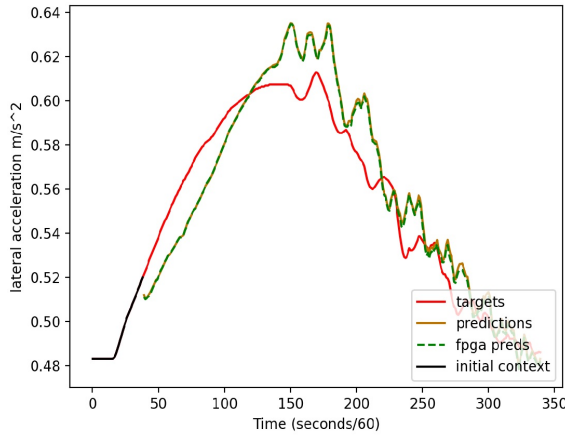
TABLE I
THE HYPERPARAMETERS OF THE PROPOSED TWO TCN.

Parameters	TCN module1	TCN module2
Input channel number	64	128
Channel length (bit)	16	16
Sequence length (bit)	1024	2048
Kernel size	3	3
Layer number	3	3
MSE loss	5.047e-4	3.0952e-4

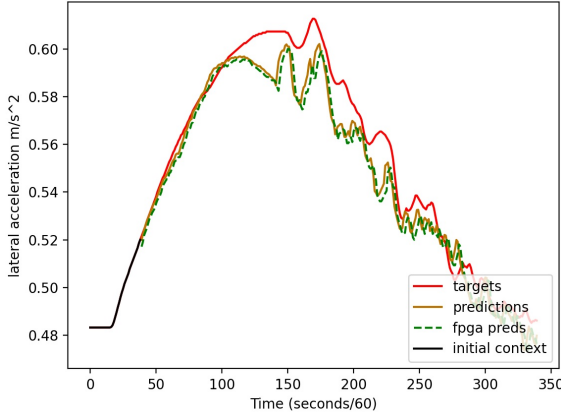
The resource requirements for the FPGA-based TCN model on a ZCU102 FPGA with different input channels number are shown in Table II and Table III. The resource requirement doubles when using 128 input channels instead of 64 input channels, but it still only uses 36% of the FPGA's resources.

V. CONCLUSION

This paper demonstrated the design of a TCN neural network-based sequence modeling, to be used as the prediction module for an ECU in a HIL simulator. The HIL



(a) Normalized lateral acceleration of truck for 64 channels.



(b) Normalized lateral acceleration of truck for 128 channels.

Fig. 5. The real behaviour of truck parameters compared to the TCN Python-based prediction values and the FPGA-based prediction values for different input channels.

TABLE II
UTILIZATION OF FPGA RESOURCES FOR 64 INPUT CHANNELS.

Resource	Utilization	Available	Utilization Percent
LUT	50468	274080	18.41
LUTRAM	3204	144000	2.23
FF	77128	548160	14.07
BRAM	102.50	912	11.24
DSP	128	2520	5.08
IO	2	328	0.61
BUFG	12	404	2.97

simulator provides an efficient and cost-effective way to test and validate complex control systems in real-world conditions. Also, an FPGA is a powerful technology that offer high-speed simulations with very low latency, making it a preferred choice for many real-time applications. The combination of HIL simulators with FPGAs can provide a powerful tool to test and validate the system in a realistic environment, reducing the time, cost, and risks associated with physical testing, and improving the reliability and safety of the control system. The experiments demonstrated the significant capability of FPGA in enhancing neural network sequence regression, achieving both

TABLE III
UTILIZATION OF FPGA RESOURCES FOR 128 INPUT CHANNELS.

Resource	Utilization	Available	Utilization Percent
LUT	99548	274080	36.32
LUTRAM	6404	144000	4.45
FF	152517	548160	27.86
BRAM	258	912	28.29
DSP	256	2520	10.16
IO	2	328	0.61
BUFG	11	404	2.72

high performance and efficient resource use.

REFERENCES

- [1] S. Chen, Y. Chen, S. Zhang, and N. Zheng, "A novel integrated simulation and testing platform for self-driving cars with hardware in the loop," *IEEE Trans. Intell. Veh.*, vol. 4, no. 3, pp. 425–436, Sep. 2019.
- [2] F. Mihalic, M. Truntic, A. Hren, *Hardware-in-the-Loop Simulations: A Historical Overview of Engineering Challenges*. Electronics 2022, 11, 2462.
- [3] X. Dai, C. Ke, Q. Quan, et al.: RFLySim: Automatic test platform for UAV autopilot systems with FPGA-based hardware-in-the-loop simulations. *Aerosp Sci Technol*. 2021; 114: 106727.
- [4] X. Guo, Y. Tang, M. Wu, Z. Zhang, J. Yuan, *Fpga-based hardware-in-the-loop real-time simulation implementation for high-speed train electrical traction system*, *IET Electric Power Applications* 14 (5)(2020) 850–858. doi:10.1049/iet-epa.2019.0655.
- [5] O. Lucia, I. Urriza, L. A. Barragan, D. Navarro, O. Jimenez, and J.M. Burdio, "Real-time FPGA-based hardware-in-the-loop simulation test bench applied to multiple-output power converters," *IEEE Trans. Ind. Appl.*, vol. 47, no. 2, pp. 853–860, Mar./Apr. 2011.
- [6] X. Dai, C. Ke, Q. Quan, K.Y. Cai, *Simulation credibility assessment methodology with FPGA-based hardware-in-the-loop platform* *IEEE Trans Ind Electron*, 68 (4) (2021), pp. 3282–3291, 10.1109/TIE.2020.2982122.
- [7] R. N. Charette, "How Software Is Eating the Car. The trend toward self-driving and electric vehicles will add hundreds of millions of lines of code to cars. Can the auto industry cope?" 2021.
- [8] A. Gupta, A. Anpalagan, L. Guan, and A. S. Khwaja, "Deep learning for object detection and scene perception in self-driving cars: Survey, challenges, and open issues," *Array*, vol. 23, Feb. 2021, Art. no. 100057.
- [9] D. Zhang and H. Li, "A stochastic-based FPGA controller for an induction motor drive with integrated neural network algorithms," *IEEE Trans. Ind. Electron.*, vol. 55, no. 2, pp. 551–561, Feb. 2008.
- [10] M. Carreras, G. Deriu, L. Raffo, L. Benini, and P. Meloni, "Optimizing temporal convolutional network inference on FPGA-based accelerators," 2020, arXiv:2005.03775.
- [11] A. Graves, "Supervised sequence labelling," in *Supervised Sequence Labelling With Recurrent Neural Networks*. Cham, Switzerland: Springer, 2012, pp. 5–13.
- [12] Y. Chen, Y. Kang, Y. Chen, and Z. Wang, "Probabilistic forecasting with temporal convolutional neural network," *Neurocomputing*, vol. 399, pp.491–501, 2020.
- [13] S. Bai, J. Zico Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv e-prints*. 2018;p. arXiv:1803.01271.
- [14] C. Lea, M. Flynn, R. Vidal, A. Reiter, and G. Hager, "Temporal convolutional networks for action segmentation and detection," *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1003–1012.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [16] CH. Barnard, P. Fortier, and A. Miled, "Evaluating the Capabilities of Temporal Convolutional Networks on a Sequence-to-Sequence Regression Task," under review.
- [17] CH. Barnard, P. Fortier, and A. Miled "An FPGA Accelerator Architecture for Temporal Convolutional Neural Networks Used as Low-Latency Sequence Model in HIL Simulator," under review.